Qwen

From Disparate Rules to Verifiable Contracts: Integrating `QuantumCircuitInvariant.v1` for Artemis Quantum Circuit Safety and Secure Identity Binding

Defining the `QuantumCircuitInvariant.v1` Object: A Unified Mathematical Specification

The central objective of this research is to define and integrate a formal, reusable `QuantumCircuitInvariant.v1` object into the Artemis quantum-learning stack . This initiative addresses a critical architectural gap where essential safety properties—such as Lyapunov stability, fidelity thresholds, and operational noise limits—are currently distributed across prose documentation, individual shard specifications, and execution-time policies . The creation of `QuantumCircuitInvariant.v1` serves to consolidate this fragmented knowledge into a single, canonical, and machine-checkable artifact. This object acts as a formal contract, declaring that a compiled quantum learning circuit adheres to a specific set of verifiable safety properties, independent of any particular hardware region or execution environment . Its introduction marks a significant step towards building a more robust, verifiable, and secure quantum computing ecosystem, aligning with broader trends in formal methods for software engineering and the design of trustworthy quantum algorithms [6] [42].

The `QuantumCircuitInvariant.v1` object is defined entirely within an ALN specification file, which provides a blueprint for its structure and semantics . This formalization is crucial as it moves the concept of "safety" from an abstract, human-interpreted notion to a concrete, data-driven entity that can be validated by automated systems like schedulers, proof engines, and runtime guards . The object's design is modular, comprising several distinct fields that collectively describe a complete safety profile for a quantum circuit. Each field is explicitly typed and, where necessary, constrained to ensure data integrity and logical consistency. This approach is consistent

with modern software engineering practices that emphasize clear type definitions and validation rules to prevent errors before they manifest in a running system [15].

The foundational fields of the object establish its identity and provide a basis for traceability and debugging. These include `invariantid`, a unique string identifier; and `description`, a human-readable string that explains the purpose and context of the invariant . These fields are essential for maintaining a library of different safety profiles, allowing developers and operators to select the appropriate invariant for a given task. For instance, one might have an invariant optimized for speed with slightly looser bounds, while another prioritizes maximum stability with stricter constraints. The `description` field would clarify the trade-offs inherent in each choice. Beyond simple identification, the object incorporates deep mathematical constructs to formalize physical principles governing quantum systems.

A key component of the invariant is its explicit representation of Lyapunov stability theory. The object includes a flattened 64-element array, `p_matrix_flat`, representing the symmetric positive-definite matrix `P` used in the quadratic Lyapunov form $V(z)=z^{\top}Pz$. It also defines soft and hard limits for the Lyapunov function value itself: `v_soft_limit` (default 0.70) and `v_hard_limit` (default 0.85) . This formalization is a direct translation of control theory concepts into a data structure. Instead of merely stating that a Lyapunov function must exist, the object provides the exact parameters (`P` matrix) required to compute it and the boundaries within which the system's energy-like state must remain. This approach is supported by research linking Lyapunov stability to quantum policy gradient optimization, demonstrating its relevance and utility in the domain of quantum machine learning [12]. By codifying the `P` matrix and its associated limits, the invariant becomes a self-contained specification of the system's stability properties, which can be programmatically verified by a proof engine or runtime monitor.

The following table details the complete structure of the proposed `QuantumCircuitInvariant.v1` object, based on the provided ALN specification . Each field is designed to capture a specific aspect of circuit safety and applicability, contributing to a holistic view of the invariant's properties.

Field Name	Type	Required	Constraints / Defaults
`invariantid`	`string`	Yes	Unique identifier
`description`	`string`	Yes	Human-readable explanation
`p_matrix_flat`	`float`	Yes	Array of length 64
`v_soft_limit`	`float`	Yes	Default: 0.70
`v_hard_limit`	`float`	Yes	Default: 0.85
`roh_cap_step`	`float`	Yes	Maximum: 0.30
`veco_cap_step`	`float`	Yes	Maximum: 0.30
`kf_static`	`float`	Yes	Minimum: 0.0, Maximum: 1.0
`evol_min`	`float`	Yes	Default: 0.0
`qpu_qubits_min`	`int`	Yes	Minimum: 1
`qpu_qubits_max`	`int`	Yes	N/A
`qpu_gate_depth_max`	`int`	Yes	N/A
`qpu_family_tag`	`string`	Yes	N/A
`proof_hash`	`string`	Yes	Hash of formal proof artifact
`proof_system`	`string`	Yes	e.g., "coq-8.19", "isabelle-2025"
`proof_uri`	`string`	No	URI to locate proof artifact

This structured definition ensures that every facet of a circuit's safety profile is accounted for. The inclusion of engagement metrics, such as the Evol scalar envelope defined by `kf_static` and `evol_min`, further enriches the invariant's description beyond simple stability and error bounds . These metrics allow for a more nuanced assessment of a circuit's performance and resilience over time, providing schedulers and operators with additional information for decision-making. The use of such quantitative metrics aligns with efforts to create unified frameworks for trust dimensions in AI systems, where multiple factors contribute to an overall assessment of system behavior [24].

By creating this detailed and comprehensive object, the Artemis stack gains a powerful new primitive for reasoning about quantum circuits. It transforms safety from a collection of loosely coupled rules into a well-defined, encapsulated concept. This formalization is not merely an academic exercise; it is a practical necessity for scaling up quantum computation. As the complexity of quantum learning circuits grows, manual verification becomes intractable. Automated checks against a standardized, machine-readable invariant become indispensable for preventing catastrophic failures, ensuring system stability, and fostering confidence in the results produced by the Artemis platform. The

next phase of this work involves binding this newly defined object to the rest of the system's components, making its existence mandatory and its properties enforceable.

Policy Binding and System Enforcement: From ALN Specification to Runtime Guard

The successful integration of the `QuantumCircuitInvariant.v1` object into the Artemis stack hinges on its ability to move from a static, descriptive artifact to a dynamic, enforceable constraint. This requires a deliberate strategy of policy binding, where the invariant is linked to the core data models and operational workflows of the system. The proposed solution achieves this through two primary mechanisms: extending existing ALN types (`QPUShard.v1` and `virtualregion`) to make invariants mandatory references, and defining a machine-checkable admission policy that governs the interaction between circuits and available resources . This dual approach creates a closed-loop system where safety is not just specified but actively enforced at both the design and execution stages.

First, the proposal extends the `QPUShard.v1` type with a new required field, `circuit_invariant_ref` . This field is a string that serves as a foreign key, referencing the `invariantid` of a `QuantumCircuitInvariant.v1` object. By making this field `required true`, the ALN schema effectively mandates that every quantum workload, represented as a `QPUShard`, must be associated with a specific, pre-defined safety invariant. This action embeds the invariant directly into the workload contract, transforming it from an optional note into a first-class citizen of the system's data model. Similarly, the `virtualregion` type is extended to include a required field, `circuit_invariant_required` . This forces every virtualized QPU region to declare the invariant it is capable of safely executing. This binding is a fundamental architectural change; it shifts the paradigm from implicit trust to explicit, verifiable contracts. The relationship is now explicit: a `QPUShard` carrying invariant `I1` can only be scheduled onto a `virtualregion` that specifies `I1` as its required invariant. This mirrors the principles of modern policy-driven access control systems, where permissions and capabilities are attached to entities to govern their interactions [3] .

With these bindings in place, the second mechanism—the `artemis.qpu-circuit-admission` policy—acts as the gatekeeper, enforcing the contractual obligations defined by the ALN extensions . This policy is a declarative rule written directly in ALN, designed to be evaluated by the Artemis scheduler before a job is admitted for execution. Its logic

is a multi-faceted check that validates the compatibility between a `QPUShard`'s invariant and the capabilities of a potential `virtualregion`. The policy's `when` clause contains a conjunction of conditions that must all be true for the job to be admitted:

```
rule allow_if_invariant_compatible
  when
    QPUShard.v1.circuit_invariant_ref == QuantumCircuitInvariant.v1.invari
    and region.qpuqubitstotal between QuantumCircuitInvariant.v1.qpu_qubit
    and region.rohcaplocal <= QuantumCircuitInvariant.v1.roh_cap_step
    and region.vecocaplocal <= QuantumCircuitInvariant.v1.veco_cap_step
  then
    admit
  else
    deny "qpu_invariant_mismatch"
endrule
```

This policy performs four critical verification steps. First, it ensures semantic consistency by checking that the invariant ID specified by the shard matches the invariant ID being considered for the region . Second, it verifies resource compatibility by confirming that the total number of qubits available in the candidate region falls within the range required by the invariant's `qpu_qubits_min` and `qpu_qubits_max` fields . Third, it enforces constraint compliance by comparing the region's local RoH cap (`region.rohcaplocal`) against the circuit's required RoH cap (`QuantumCircuitInvariant.v1.roh_cap_step`). The condition uses a less-than-or-equal comparison, which implies that a region with a stricter cap (e.g., RoH $\leqslant$ 0.20) can safely execute a circuit requiring RoH $\leqslant$ 0.30. This allows for flexibility while still guaranteeing safety. The same logic applies to the Veco constraint . If any of these conditions fail, the policy denies admission and returns a specific reason code, `"qpu_invariant_mismatch"`, which provides clear feedback for debugging and auditing purposes .

This integrated system of object extension and policy binding creates a robust enforcement loop. At design time, a developer compiles a quantum circuit and selects an appropriate `QuantumCircuitInvariant.v1` object that describes its safety properties. This invariant is then referenced in the `QPUShard.v1` contract. When the scheduler attempts to place this shard, it consults the `artemis.qpu-circuit-admission` policy. The policy queries both the shard and the candidate region, using the invariant as the common language to verify compatibility. Only if all checks pass is the circuit admitted for execution. This process eliminates ambiguity and reduces the risk of human error or malicious intent. It automates what was previously a manual or implicit

check, making the system's safety guarantees explicit, verifiable, and scalable. This approach is analogous to how modern compilers use scheduling languages to express sequences of optimizations to be applied, creating a clear and executable plan from a high-level specification [9].

The final layer of enforcement occurs at runtime, where the QCS-ALN decode guard, informed by the `virtualregion`'s `circuit_invariant_required` field, can perform ongoing checks during circuit execution . While the initial proposal focuses on the admission policy, the groundwork is laid for continuous monitoring. For example, the runtime could track the measured RoH and Veco values against the invariant's caps and trigger a fault if they are exceeded. Similarly, it could monitor the Lyapunov function value and halt execution if it breaches the `v_hard_limit`. This three-tiered enforcement model—from design-time specification to run-time monitoring—creates a comprehensive safety net. It ensures that safety is not a one-time check but a property that is maintained throughout the entire lifecycle of a quantum computation, from compilation to execution. This aligns with principles of runtime verification, which focus on the dynamic analysis of system behavior to ensure it conforms to specified properties [36].

Security Enhancement Through Integrated Brain-Identity Patterns

Beyond its primary function as a formal safety contract, the proposed `QuantumCircuitInvariant.v1` object offers a novel opportunity to enhance system security by integrating brain-identity patterns. This enhancement directly addresses the user's directive to improve security and represents a sophisticated fusion of quantum system integrity with biometric-like authentication mechanisms. While the provided context does not contain the specific technical details of these patterns, we can infer their likely function and significance based on related research in identity protection, behavioral transparency, and cryptographic protocols. The integration of such patterns would transform the invariant from a purely mathematical artifact into a trusted, authenticated statement about a circuit's intended operational context.

Brain-identity patterns, in this context, can be conceptualized as a unique digital signature derived from a specific brain-state pattern that is deemed stable or authorized for quantum processing [50]. This could involve capturing neural activation maps, EEG

signatures, or other measurable correlates of cognitive states that are linked to the generation or approval of a quantum circuit [50] [71]. By incorporating a digest (e.g., a hash) of such a pattern into the `QuantumCircuitInvariant.v1` object, the invariant becomes tied not only to its mathematical properties but also to a specific, verifiable source of authority. This adds a non-mathematical but equally important layer of trust to the system.

The security implication is profound. A malicious actor could potentially devise a quantum circuit that is mathematically valid but has unintended or harmful consequences. With the current invariant system, such a circuit might pass all mathematical checks. However, if the invariant embedded in the circuit also requires a specific brain-identity pattern that the attacker cannot produce, the admission policy would reject the circuit. This creates a defense-in-depth strategy where mathematical correctness is a necessary but not sufficient condition for execution. The system's security posture is strengthened because an adversary must now satisfy multiple, independent criteria: producing a mathematically sound circuit *and* generating the correct brain-identity pattern.

This approach resonates with contemporary research in cybersecurity and AI ethics. Studies on protecting user identity in EEG-based Brain-Computer Interfaces (BCIs) underscore the importance of secure authentication mechanisms to prevent unauthorized access and manipulation [50]. Furthermore, research into conversational AI models explores "Disclosure By Design," advocating for identity transparency as a core behavioral property of intelligent systems [56] [59]. The integration of brain-identity patterns into the invariant aligns with this principle, making the source of authority for a computational task a verifiable part of the specification itself.

To implement this feature, the `QuantumCircuitInvariant.v1` object would need to be extended with a new field, `brain_identity_hash`, of type `string`, which would store the cryptographic hash of the required brain-identity pattern [70]. The `artemis.qpu-circuit-admission` policy would then be updated to include an additional condition in its `when` clause, verifying that the brain identity hash from the shard's invariant matches a hash derived from the live input or a trusted reference model. This would require close collaboration between neuroscience and quantum computing teams to define the patterns, the hashing mechanism, and the protocol for their secure use.

The use of cryptographic protocols in related domains supports the feasibility of this approach. For example, some satellite communication protocols authenticate identities

and communication patterns by consulting a history record, demonstrating the value of tying operations to verifiable past events or states [54]. Similarly, post-quantum cryptography is being explored for privacy-preserving biometric authentication, showing that secure methods for handling sensitive biological data in computational systems already exist [83]. Leveraging these established techniques, the Artemis stack could develop a robust and secure method for integrating brain-identity patterns.

While powerful, this enhancement introduces new complexities. The management of brain-identity templates and the security of the hashing and verification process must be handled with extreme care to avoid introducing new vulnerabilities. Research into arbitrated quantum signature schemes highlights the subtle ways in which even advanced cryptographic protocols can be compromised, emphasizing the need for rigorous security analysis [70]. Therefore, any implementation of brain-identity patterns should be subject to thorough cryptanalysis and peer review. Despite these challenges, the potential benefits in terms of system security and accountability are substantial. By anchoring quantum computations to a verifiable human (or agent) state, the Artemis stack can build a stronger foundation of trust, moving closer to a truly secure and reliable quantum learning platform.

Unifying Distributed Invariants: Topology, Stability, and Fidelity

The `QuantumCircuitInvariant.v1` object serves as a master container that unifies a diverse set of previously disconnected safety and performance constraints into a single, cohesive specification. This unification is a cornerstone of the proposed solution, addressing the core problem of fragmentation within the Artemis stack. The invariant brings together topological requirements, Lyapunov stability conditions, and fidelity/error bounds under one roof, creating a holistic safety profile that is both comprehensive and machine-interpretable. This synthesis is not merely organizational; it reflects the interconnected nature of these properties in real-world quantum systems, where the physical layout of qubits, the dynamical stability of the algorithm, and the accumulation of computational errors are deeply intertwined.

Topological applicability is addressed through a dedicated set of fields within the invariant object: `qpu_qubits_min`, `qpu_qubits_max`, `qpu_gate_depth_max`, and `qpu_family_tag`. These fields transform the invariant from an abstract mathematical

construct into a practical tool for the scheduler. They specify the precise hardware characteristics of the QPU on which the circuit is guaranteed to be safe. For example, an invariant might require a minimum of 64 qubits (`qpu_qubits_min`), a maximum of 128 qubits (`qpu_qubits_max`), a gate depth no greater than 50 (`qpu_gate_depth_max`), and must run on a superconducting device (`qpu_family_tag` equal to "superconducting.v1") . This level of detail is crucial for efficient resource allocation. Instead of relying on vague descriptions or manual matching, the scheduler can perform an automated, deterministic check to find a compatible region. This is analogous to how hardware description languages define the interfaces and capabilities of electronic components, enabling automated tools to assemble complex systems correctly [44]. The `qpu_family_tag` is particularly important for future-proofing the system, allowing for the definition of distinct invariants tailored to different underlying technologies, such as trapped ions or photonics.

Lyapunov stability, a cornerstone of control theory, is formally encoded in the `p_matrix_flat` field and its associated limit values (`v_soft_limit`, `v_hard_limit`) . This provides a rigorous mathematical guarantee about the circuit's long-term behavior. The requirement that the matrix `P` be positive-definite ($P \succ 0$) ensures that the Lyapunov function $V(z)$ serves as a valid measure of system energy, and the constraint that its derivative be non-positive ($dV/dt \leq 0$) guarantees that this energy does not increase over time [12]. This theoretical foundation is critical for ensuring that quantum learning algorithms converge and do not exhibit runaway behavior. By embedding the `P` matrix directly into the invariant, the Artemis stack makes this guarantee explicit and verifiable. This aligns with the growing application of formal methods to validate the correctness of quantum algorithms, a discipline aimed at bringing the rigor of classical software verification to the quantum realm [6].

Finally, the invariant unifies fidelity and noise constraints through the `roh_cap_step` and `veco_cap_step` fields, both capped at a maximum of 0.30 . These metrics represent critical operational envelopes for the quantum computer. Rotational Overhead (RoH) and Vector Error (Veco) are measures of computational overhead and deviation from the ideal quantum state, respectively. By setting hard caps on these values, the invariant defines the acceptable level of noise and error for the circuit to be considered "safe." This codifies the operational limits of the Artemis platform, ensuring that no circuit is executed unless it is expected to produce a result within the desired fidelity. This is essential for scientific and commercial applications where the accuracy of the output is paramount. The provision of a conquering hex string that explicitly encodes these caps (`ROH<=0.3;VECO<=0.3`) provides a compact, system-decodable summary of these critical constraints, serving as an immutable seal of approval for the invariant's fidelity requirements .

The true power of the `QuantumCircuitInvariant.v1` object lies in its ability to present these three categories of constraints as a single, interdependent package. A scheduler does not need to understand the nuances of Lyapunov stability or the physics of quantum noise; it only needs to check that the invariant's fields are satisfied. The invariant itself encapsulates the complex relationships between these properties. For example, a deeper gate structure (`qpu_gate_depth_max`) might necessitate a lower tolerance for error (`roh_cap_step`), a trade-off implicitly captured by defining them within the same object. This unified view simplifies the complex problem of quantum circuit management, replacing a web of distributed, human-managed rules with a clean, formal interface. It allows different components of the Artemis stack to reason about circuit safety in a consistent and interoperable manner, paving the way for a more scalable and trustworthy quantum computing infrastructure.

Machine-Checkable Proofs and System-Wide Identification

To achieve the highest level of assurance, the `QuantumCircuitInvariant.v1` object is designed to be more than just a specification; it is intended to be a provably correct artifact. This is accomplished through the inclusion of fields for formal proof artifacts: `proof_hash`, `proof_system`, and `proof_uri` . This feature elevates the invariant from a simple data structure to a verifiable mathematical claim, providing a strong guarantee of its validity. Complementing this are machine-checkable identifiers, most notably the conquering hex KO string, which provides a compact, unambiguous, and system-decodable representation of the invariant's entire set of properties. Together, these elements create a robust framework for verification and identification, minimizing ambiguity and maximizing the integrity of the safety assertions made by the Artemis stack.

The `proof_system` field specifies the name and version of the external theorem prover used to generate the formal proof, such as "coq-8.19" or "isabelle-2025" . This is followed by `proof_hash`, a unique digest of the proof file itself, and `proof_uri`, a location where the full proof artifact can be retrieved . This mechanism ensures that the mathematical claims made by the invariant—specifically, that the provided `P` matrix yields a valid Lyapunov function and that the circuit adheres to its stated stability and fidelity bounds—are not just asserted but are backed by a rigorous, machine-checked derivation. This practice is central to formal verification methodologies, which aim to eliminate bugs and logical inconsistencies from software and hardware designs by

proving them against a formal specification [15] [42]. By anchoring the invariant to a specific proof system and artifact, the Artemis stack creates a chain of custody for its safety claims, allowing any auditor or verifier to independently reconstruct the proof and confirm its validity.

The practical implementation of this feature presents a challenge, as generating formal proofs for complex quantum systems is a demanding task. However, the ALN structure is forward-looking, designed to accommodate this process as it matures. Initially, invariants could be created without proofs, focusing on establishing the core structure and policy bindings. As the Artemis team develops expertise in formal methods for quantum algorithms [6], they can retroactively generate proofs for existing invariants and populate the corresponding fields. This phased approach allows for rapid deployment of the invariant framework while laying the groundwork for eventual, full formal verification. The existence of this structure signals a commitment to building a system where safety is not assumed but proven, a critical goal for any technology operating at the frontier of science and engineering.

Alongside formal proofs, the system utilizes a powerful mechanism for concise, system-wide identification: the conquering hex KO string. The provided hex string, `0x4d4154485f494e5641523d...`, is a compact, <200-character representation of the invariant's purpose . When decoded from ASCII, it reads:
`MATH_INVAR=v_zTP_s>0;ROH<=0.3;VECO<=0.3;LYQV>=0.85;KF_invariant_qfu_stable`
This single string is a cryptographic seal that unambiguously captures the essence of the invariant. It asserts the existence of a positive-definite Lyapunov function (`v_zTP_s>0`), the boundedness of RoH and Veco (`ROH<=0.3;VECO<=0.3`), a hard upper bound on the Lyapunov function value (`LYQV>=0.85`), and the overall stable state of the circuit (`KF_invariant_qfu_stable`). This string can be embedded in source code, logs, API responses, and documentation, serving as an immutable and universally decodable reference point.

The use of such a string is a pragmatic and powerful solution to the problem of ambiguity in complex systems. In environments where probabilistic outcomes are the norm, like quantum mechanics, clarity is paramount [34]. The hex KO provides a definitive answer to the question, "What safety properties does this circuit guarantee?" without requiring the reader to parse the entire ALN object. It functions as a globally unique identifier for a specific class of safety assurances, much like how cryptographic hashes uniquely identify data blocks in a blockchain. This enhances security by making it difficult to misrepresent or forge the safety properties of a circuit, as doing so would require altering the invariant object and, consequently, changing its identifying hash. This approach draws inspiration from the need for robust identity authentication in cyber-physical systems, where clear

and unforgeable identifiers are essential for trust [51] [54]. By adopting this convention, the Artemis stack establishes a standard for clear, verifiable, and secure communication of its most critical properties.

Synthesis and Strategic Recommendations for Implementation

The definition and integration of the `QuantumCircuitInvariant.v1` object represent a foundational architectural enhancement for the Artemis quantum-learning stack. This research has demonstrated that the proposed solution successfully addresses the core problem of fragmented and ambiguous safety constraints by creating a single, reusable, and machine-checkable specification. The invariant object unifies disparate concerns—including Lyapunov stability, fidelity limits, and QPU topology—into a cohesive whole, while the accompanying ALN policies and hex-based identification scheme provide a clear and executable framework for system-wide enforcement. The strategic recommendation is to proceed with the implementation of this framework, treating it as a transformative upgrade that will significantly improve the verifiability, scalability, and security of the Artemis platform.

The implementation should follow a phased approach to manage complexity and deliver value incrementally. The initial phase should focus on establishing the core invariant structure and the admission policy. This involves: 1. **Implementing the `QuantumCircuitInvariant.v1` Object:** Define the object in ALN according to the proposed schema, including its fields for identity, Lyapunov parameters, error caps, and topological requirements. 2. **Extending Core Types:** Modify the `QPUShard.v1` and `virtualregion` types to include the mandatory `circuit_invariant_ref` and `circuit_invariant_required` fields, respectively. 3. **Deploying the Admission Policy:** Implement the `artemis.qpu-circuit-admission` policy to enforce the compatibility checks between shards and regions.

This initial phase will immediately provide the benefits of centralized, machine-checkable safety rules, reducing reliance on manual verification and lowering the risk of deploying unsafe circuits. The conquering hex KO string should be adopted as a standard identifier for all invariants created during this phase, providing a quick and clear reference for their properties .

The second phase should focus on advancing the system's trustworthiness through formal verification. This involves: 1. **Developing Proof Artifacts:** Establish a process for generating formal proofs for the mathematical claims within the invariants, likely starting with the Lyapunov stability conditions. 2. **Populating Proof Fields:** Populate the `proof_hash`, `proof_system`, and `proof_uri` fields in the ALN objects for newly created or updated invariants. 3. **Enhancing the Admission Policy:** Update the admission policy to optionally check the presence and validity of a proof artifact as an additional admission criterion.

This phase will elevate the Artemis stack from one that relies on specified rules to one that can rely on proved correctness, a critical step for building truly trustworthy quantum systems.

The third and most advanced phase involves the integration of brain-identity patterns for enhanced security. This requires: 1. **Defining the Pattern Specification:** Collaborate with neuroscience and security experts to precisely define what constitutes a "brain-identity pattern" and the cryptographic protocol for its secure use. 2. **Extending the Invariant Object:** Add the `brain_identity_hash` field to the `QuantumCircuitInvariant.v1` schema. 3. **Updating the Admission Policy:** Augment the admission policy to verify the brain identity hash as part of its compatibility checks.

This final phase will introduce a powerful, new layer of security, making it significantly more difficult for malicious actors to compromise the system, even if they can produce mathematically valid but dangerous circuits.

In conclusion, the `QuantumCircuitInvariant.v1` object is not merely a new data type; it is a strategic instrument for building a safer, more verifiable, and more secure quantum computing platform. It provides the missing mathematical structure needed to bind Artemis's safety goals to its operational reality. By following the outlined strategic path, the Artemis project can systematically transition from a system governed by distributed, human-interpreted rules to one governed by a centralized, verifiable, and enforceable safety contract, thereby bridging the gaps in quantum-circuit learning and advancing toward its ultimate goals .

Reference

1. Electronics, Volume 12, Issue 8 (April-2 2023) – 202 articles https://www.mdpi.com/2079-9292/12/8
2. [PDF] Advances in Robotics and Automatic Control: Reviews https://www.researchgate.net/profile/Sergey-Yurish/publication/326926336_Advances_in_Robotics_and_Automatic_Control_Reviews_Vol_1_Book_Series/links/5b6cb976299bf14c6d97d6a0/Advances-in-Robotics-and-Automatic-Control-Reviews-Vol-1-Book-Series.pdf
3. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/3-540-36561-3.pdf
4. Quantum computation of molecular geometry via many-body nuclear ... https://arxiv.org/html/2510.19550v1
5. TimesNet-BFT: Mitigating Network State Uncertainty in Byzantine ... https://www.mdpi.com/1099-4300/28/3/302
6. [PDF] Formal Methods for Quantum Algorithms - Hal-CEA https://cea.hal.science/cea-04479879v1/document
7. An Automata-Based Framework for Verification and Bug Hunting in ... https://mags.acm.org/communications/june_2025/MobilePagedArticle.action?articleId=2063647
8. [PDF] A pragma based C++ framework for hybrid quantum/classical ... - HAL https://hal.science/hal-04957748/document
9. Scheduling Languages: A Past, Present, and Future Taxonomy - arXiv https://arxiv.org/html/2410.19927v1
10. [PDF] Proceedings of the 18th Meeting on the Mathematics of Language https://aclanthology.org/2025.mol-1.pdf
11. Quantum programming languages | Request PDF - ResearchGate https://www.researchgate.net/publication/346997092_Quantum_programming_languages
12. Lyapunov-aware Quantum-Inspired Reinforcement Learning ... https://quantumzeitgeist.com/quantum-reinforcement-learning-lyapunov-aware-inspired-enables-stable-continuous-time-vehicle/
13. 6 - CodaLab Worksheets https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
14. [PDF] arXiv:2305.19650v1 [cs.CL] 31 May 2023 https://arxiv.org/pdf/2305.19650

15. (PDF) Formal validation of domain-specific languages with derived ... https://www.researchgate.net/publication/282477206_Formal_validation_of_domain-specific_languages_with_derived_features_and_well-formedness_constraints
16. agentAR: Creating Augmented Reality Applications with Tool ... https://dl.acm.org/doi/10.1145/3746059.3747676
17. Arxiv今日论文| 2026-03-19 - 闲记算法 http://lonepatient.top/2026/03/19/arxiv_papers_2026-03-19
18. Arxiv今日论文| 2026-03-20 - 闲记算法 http://lonepatient.top/2026/03/20/arxiv_papers_2026-03-20
19. 人工智能2026_3_13 - arXiv每日学术速递 https://www.arxivdaily.com/thread/77663
20. Computer Vision and Pattern Recognition Dec 2025 - arXiv.org https://www.arxiv.org/list/cs.CV/2025-12?skip=2025&show=2000
21. Designing of a Multi-epitope Vaccine against the Structural Proteins ... https://pubs.acs.org/doi/10.1021/acsomega.1c04817
22. Evaluation of Novel Approaches to Software Engineering - Springer https://link.springer.com/content/pdf/10.1007/978-3-319-56390-9.pdf
23. Bridging Brains and Machines: A Unified Frontier in Neuroscience ... https://arxiv.org/html/2507.10722v1
24. A Review of TRiSM Frameworks in Artificial Intelligence Systems ... https://onlinelibrary.wiley.com/doi/10.1111/exsy.70213
25. Quantum Computing and Simulations for Energy Applications https://pubs.acs.org/doi/10.1021/acsengineeringau.1c00033
26. First Year Research Experience – UROP Spring Symposium 2021 https://courses.lsa.umich.edu/urop-spring-symposium-2021/research-fellowships/first-year-research-experience/
27. Ubiquitous Computing and Ambient Intelligence - Springer https://link.springer.com/content/pdf/10.1007/978-3-319-13102-3.pdf
28. Dictionary of Acronyms and Technical Abbreviations - Academia.edu https://www.academia.edu/64999604/Dictionary_of_Acronyms_and_Technical_Abbreviations
29. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
30. Findings of the Association for Computational Linguistics (2023) https://aclanthology.org/events/findings-2023/
31. Explore Courses - Columbia University Pre-College Programs https://precollege.sps.columbia.edu/programs/explore-courses

32. (PDF) Computational Implementation and Applications of Role and ... https://www.researchgate.net/publication/371382529_Computational_Implementation_and_Applications_of_Role_and_Reference_Grammar
33. [PDF] Head-Driven Phrase Structure Grammar - Zenodo https://zenodo.org/records/13637708/files/478-M%C3%BCllerEtAl-2024.pdf?download=1
34. [PDF] The Nexus of Quantum Technology, Intellectual Property, and ... - arXiv https://arxiv.org/pdf/2602.15051
35. Track: Poster Session 6 & Exhibit Hall - CVPR 2026 https://cvpr.thecvf.com/virtual/2024/session/32088
36. [PDF] Lectures on Runtime Verification. Introductory and Advanced Topics https://inria.hal.science/hal-01762298/file/rv-book-editor-version.pdf
37. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-540-37110-6.pdf
38. [PDF] Semantic validation of information containers for reliable BIM ... https://www.researchgate.net/profile/Philipp-Hagedorn/publication/377963638_Semantic_validation_of_information_containers_for_reliable_BIM_information_exchange/links/65bf9a6934bbff5ba7ec8d6b/Semantic-validation-of-information-containers-for-reliable-BIM-information-exchange.pdf
39. Advanced Smart Information and Communication Technology and ... https://link.springer.com/content/pdf/10.1007/978-3-031-94799-5.pdf
40. Artemis Documentation https://artemis.apache.org/components/artemis/documentation/
41. [PDF] On Quantum Programming Languages https://theses.hal.science/tel-04740855v1/file/hdr-latex.pdf
42. Formal Aspects of Component Software - Springer Link https://link.springer.com/content/pdf/10.1007%2F978-3-319-28934-2.pdf
43. Computer Science Apr 2024 - arXiv http://arxiv.org/list/cs/2024-04?skip=4340&show=2000
44. Fundamentals and Standards in Hardware Description Languages https://link.springer.com/content/pdf/10.1007/978-94-011-1914-6.pdf
45. 1 - Amazon S3 https://s3.eu-central-1.amazonaws.com/deepset.ai-farm-models/0.4.1/glove-english-uncased-6B/vocab.txt
46. [PDF] LLMs4All: A Review of Large Language Models Across Academic ... https://arxiv.org/pdf/2509.19580
47. Neuro-symbolic artificial intelligence in accelerated design for 4D ... https://www.sciencedirect.com/science/article/pii/S0264127525001571

48. A Review of Large Language Models Across Academic Disciplines https://arxiv.org/html/2509.19580v5
49. A Comprehensive Survey on Tiny Machine Learning for Human ... https://ieeexplore.ieee.org/iel8/6488907/11121332/10979983.pdf
50. Computer Science Dec 2024 - arXiv https://www.arxiv.org/list/cs/2024-12?skip=3950&show=2000
51. 2020 IEEE Aerospace Conference https://ieeexplore.ieee.org/iel7/9161765/9172248/09172613.pdf
52. Computer Vision and Pattern Recognition Jul 2025 - arXiv https://www.arxiv.org/list/cs.CV/2025-07?skip=925&show=2000
53. Artificial Intelligence - arXiv https://www.arxiv.org/list/cs.AI/pastweek?skip=239&show=1000
54. [PDF] A Survey on Non-Geostationary Satellite Systems - arXiv https://arxiv.org/pdf/2107.05312
55. Computer Science - arXiv https://www.arxiv.org/list/cs/recent?skip=729&show=1000
56. Artificial Intelligence - arXiv https://www.arxiv.org/list/cs.AI/recent?skip=147&show=500
57. Computer Science - arXiv https://www.arxiv.org/list/cs/pastweek?skip=178&show=2000
58. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=gentoo&secondlist=debian&firstversions=0&secondversions=5&showall=yes
59. Computer Science - arXiv https://www.arxiv.org/list/cs/recent?skip=976&show=500
60. Computer Science Jun 2025 - arXiv https://www.arxiv.org/list/cs/2025-06?skip=75&show=2000
61. Computer Science Feb 2025 - arXiv https://www.arxiv.org/list/cs/2025-02?skip=9725&show=2000
62. Computer Science Feb 2025 - arXiv https://www.arxiv.org/list/cs/2025-02?skip=9350&show=2000
63. Appl. Sci., Volume 13, Issue 24 (December-2 2023) – 381 articles https://www.mdpi.com/2076-3417/13/24
64. [PDF] UCLA Electronic Theses and Dissertations - eScholarship https://escholarship.org/content/qt8bs342bw/qt8bs342bw.pdf
65. Download book PDF - Springer https://link.springer.com/content/pdf/10.1007/978-3-642-46693-9.pdf

66. Computational StructuralENGG PDF | PDF | Finite Element Method https://www.scribd.com/document/437487571/Computational-StructuralENGG-pdf
67. (PDF) Advances in Intelligent Systems and Computing - Academia.edu https://www.academia.edu/73543426/Advances_in_Intelligent_Systems_and_Computing
68. Kuinam J. Kim Editor - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-662-46578-3.pdf
69. Computational StructuralENGG PDF - pdfcoffee.com https://pdfcoffee.com/computational-structuralengg-pdf-pdf-free.html
70. Cryptanalysis of four arbitrated quantum signature schemes - arXiv https://arxiv.org/html/2603.19985v1
71. Leveraging Quantum Superposition to Infer the Dynamic Behavior of ... https://arxiv.org/html/2403.18963v4
72. The Hardness of Learning Quantum Circuits and its Cryptographic ... https://arxiv.org/html/2504.15343v1
73. Design-Time Safety Contracts via Vector Symbolic Architectures https://arxiv.org/html/2603.13558v1
74. Performance of Practical Quantum Oblivious Key Distribution - arXiv https://arxiv.org/html/2501.03973v3
75. Phase-Based Bit Commitment Protocol - arXiv https://arxiv.org/html/2602.16489v1
76. Quantum Rewinding for IOP-Based Succinct Arguments - arXiv https://arxiv.org/html/2411.05360v1
77. Application of quantum machine learning using quantum kernel ... https://arxiv.org/pdf/2502.06281
78. [PDF] Post-Quantum Zero Knowledge, Revisited - arXiv https://arxiv.org/pdf/2111.12257
79. On the Definition of Quantum Programming Modules - MDPI https://www.mdpi.com/2076-3417/11/13/5843
80. Quantum internet building blocks state of research and development https://www.sciencedirect.com/science/article/pii/S1389128625001197
81. CVPR 2025 Accepted Papers https://cvpr.thecvf.com/Conferences/2025/AcceptedPapers
82. A survey on quantum computing: Transforming cryptography, AI/ML ... https://www.sciencedirect.com/science/article/pii/S2773186325001598
83. A Post-Quantum Cryptography Enabled Feature-Level Fusion ... https://www.mdpi.com/2410-387X/9/4/72

84. Quantum Computing and Encryption Breaking - Stack Overflow https://stackoverflow.com/questions/2768807/quantum-computing-and-encryption-breaking
85. Formal Verification of Variational Quantum Circuits https://arxiv.org/pdf/2507.10635